

Your Python Toolkit and Environment Setup

Chapter at a glance

Before you start: Create folders and run basic **terminal commands**.

By the end, you can: organize data and notebooks, configure an environment, install dependencies, and run labs reproducibly.

1.1 The course project layout

Use UTP-MACHINE-LEARNING/ as the project root, with these folders:

Datasets

```
all_datasets/  
  lab01-03_airports.csv  
  lab01-03_routes.csv  
  lab04-AirQuality.csv  
  lab05-sensor_readings_24.csv  
  lab06-ocr-handwritten-prescription.zip  
  lab07-Tweets.csv  
  lab08-insurance.csv
```

Experiments, helper code, and generated evidence

```
all_experiments/  
  lab01_search_foundations_worked.ipynb  
  lab02_informed_search_worked.ipynb  
  lab03_search_engineering_worked.ipynb  
  lab04_fuzzy_anfis_worked.ipynb  
  lab05_robotics_sensor_control_worked.ipynb  
  lab06_handwriting_ocr_worked.ipynb  
  lab07_airline_sentiment_worked.ipynb  
  lab08_genetic_algorithm_regression_worked.ipynb  
utils/  
  core.py  
  openflights.py  
  plotting.py  
  __main__.py  
  test_search.py  
  search_queries.csv  
results/  
  lab01/ ... lab08/  
requirements.txt
```

Key idea

The notebooks import reusable search code from `utils/`. `results/` stores generated evidence and should not be edited by hand; `requirements.txt` lists the packages installed in `.venv/`.

Keep the digits ZIP intact. Each lab has one worked notebook; `predict_passenger.py` uses the saved Lab 2 model. Notebooks write models and cards to `models/` and evidence to `results/`; do not edit generated outputs.

1.2 Download the datasets

Click a **dataset name** to download it, then extract the files into **all_datasets/**.

Kaggle dataset	Lab	Use
OpenFlights: Routes & Airports	1–3	Route search, locations and distances
Air Quality	4	Fuzzy logic and ANFIS
Wall-Following Robot	5	Sensor-based control
Doctor Handwriting	6	OCR evaluation
Twitter Airline Sentiment	7	Sentiment classification
Medical Cost	8	GA regression

1.2.1 Download steps

Step 1: Open the link, click **Download**, and **extract** the archive.

Step 2: Copy the required **CSV files or images** into **all_datasets/**.

Step 3: Check that your notebook's **file paths** match the filenames, including **capitalization**.

Common mistake

For **FileNotFoundError**, check the working directory and filename. Fix the path rather than moving files into the project root.

1.3 Two ways to set up: plain Python or Anaconda

Key idea

Use one environment per project; activate it before installing or running packages.

1.4 Route A: install Python and configure PATH

Download Python from the **official downloads page**.

Windows Run the installer, enable **PATH**, and reopen the terminal. Use **py**.

macOS Run the **.pkg installer** and restart Terminal. Keep the **system Python** unchanged.

Linux Install **Python 3** with your package manager. Do not replace the **system Python**.

Windows PATH fix. If **py** is missing, open **Edit environment variables for your account**, then **Path > Edit**. Add the installer's command directory, for example:

```
%LocalAppData%\Python\bin
```

macOS PATH fix. If the installer placed Python in `/usr/local/bin` but your terminal cannot find it, add that directory to your Z shell's **PATH**:

```
echo 'export PATH="/usr/local/bin:$PATH"' >> ~/.zshrc
source ~/.zshrc
```

For **Debian or Ubuntu**:

```
sudo apt update
sudo apt install python3 python3-venv python3-pip
```

Verify the installation and executable location:

```
# Windows
py --version
py -m pip --version
where py

# macOS or Linux
python3 --version
python3 -m pip --version
which -a python3
```

1.5 Route A: create an isolated virtual environment

venv keeps this project's packages separate from other projects. See the **venv documentation**.

The root-level **requirements.txt** lists dependencies, including PyTorch (torch) for Labs 7–8. A CPU is sufficient.

```
numpy
pandas
scipy
scikit-learn
matplotlib
jupyter
nbformat
nbclient
nbconvert
pillow
pytesseract
pytest
```

Create the environment in the **project root**:

```
# Windows
py -m venv .venv

# macOS or Linux
python3 -m venv .venv
```

Activate it from the same folder:

```
# Windows (PowerShell)
.venv\Scripts\Activate.ps1

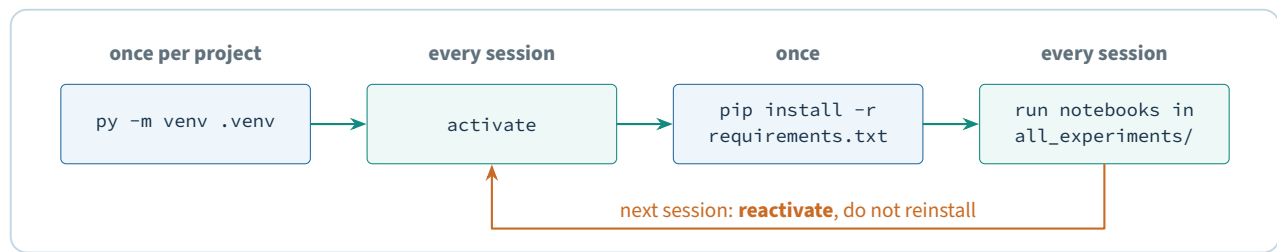
# macOS/Linux
source .venv/bin/activate
```

Install dependencies after activation:

```
# Windows
py -m pip install --upgrade pip
py -m pip install -r requirements.txt

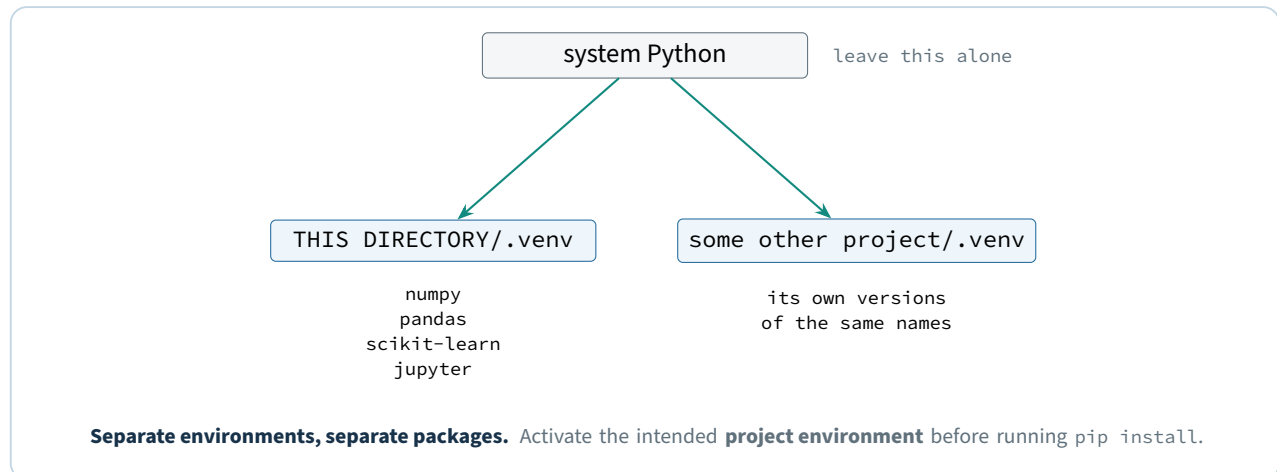
# macOS or Linux
python3 -m pip install --upgrade pip
python3 -m pip install -r requirements.txt
```

Install once; activate every session. Reuse **.venv** without reinstalling unchanged requirements.



Key idea

Activate first, then install. Keep course packages inside the **project environment**, not in a shared global installation.



1.6 Route B: Anaconda or Miniconda

Miniconda supplies Python and conda; **Anaconda** adds Navigator and preinstalled packages. Download **Miniconda** or **Anaconda**.

Windows Use default installer options without adding conda to PATH. Open **Anaconda Prompt** from Start.

macOS / Linux Run the `.pkg` or `.sh` installer, then close and reopen the terminal. The prompt shows `(base)`, which means conda is active.

Create one environment for the course, with a Python version the libraries support, then **activate** it:

```
conda create -n ml python=3.12
conda activate ml
```

Install dependencies with the same `requirements.txt` used in Route A:

```
pip install -r requirements.txt
```

Create once, activate each session, and avoid installing into `(base)`. Check the active environment and Python:

```
conda env list
python -c "import sys; print(sys.executable)"
```

Common mistake

Avoid installing the same package with both conda and pip. Use pip with the course requirements. Recreate a broken environment with `conda env remove -n ml`, then repeat setup.

Troubleshooting

Conda not found on Windows: use Anaconda Prompt, or initialize PowerShell there with `conda init powershell`. **Jupyter import failure:** activate `ml`, restart Jupyter, and check the kernel with Lab 1's environment cell.

1.7 Set up Git and GitHub for submission

The **official Git download page** lists current installers and package-manager commands.

1.7.1 Install Git

Use the command for your operating system, then reopen the terminal:

```
# Windows PowerShell
winget install --id Git.Git -e --source winget

# macOS: install Apple's command-line tools
brew install git

# Linux
sudo apt update
sudo apt install git
```

On Windows, use **Git Bash** for the SSH commands below. Confirm the installation on any system:

```
git --version
```

1.7.2 Configure your commit identity

Set the name and email that should appear on future commits. Use an email connected to your GitHub account, or your GitHub-provided private email.

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
git config --global init.defaultBranch main

git config --global --list
```

1.7.3 Create an SSH key for GitHub

SSH lets Git authenticate without asking for a GitHub password on every push. First check whether this computer already has a key:

```
ls -al ~/.ssh
```

If `id_ed25519` and `id_ed25519.pub` do not exist, create them. Enter the email connected to GitHub and choose a passphrase when prompted:

```
ssh-keygen -t ed25519 -C "you@example.com"
```

Press Enter to accept the default file location. Then load the *private* key into the SSH agent:

```
# Windows Git Bash or Linux
```

```
eval "$(ssh-agent -s)"
ssh-add ~/.ssh/id_ed25519

# macOS
eval "$(ssh-agent -s)"
ssh-add --apple-use-keychain ~/.ssh/id_ed25519
```

Copy the *public* key. Never upload or share the file without `.pub`.

```
# Windows Git Bash
clip < ~/.ssh/id_ed25519.pub

# macOS
pbcopy < ~/.ssh/id_ed25519.pub

# Linux: display it, then copy the complete line
cat ~/.ssh/id_ed25519.pub
```

In GitHub, open **Settings > SSH and GPG keys > New SSH key**. Give the key a name such as *Course laptop*, paste it, and select **Authentication Key**. The full procedure is maintained in the **GitHub SSH guide**

1.7.4 Video walkthroughs for SSH setup

Use the walkthrough for your operating system if you prefer to follow the SSH setup on screen:

Choose your operating system

Linux	Set up GitHub SSH on Linux
Windows	Set up GitHub SSH on Windows
macOS	Set up GitHub SSH on macOS

Do not copy another person's email address, key, username, or repository name from the video; replace each example with your own details.

1.7.5 Prepare the course repository

Do not upload the virtual environment, passwords, tokens, or API keys. In the project root, create a file named `.gitignore` containing:

```
.venv/
__pycache__/
*.pyc
.ipynb_checkpoints/
.env
```

Create a new repository on GitHub. Use a name such as `utp-machine-learning`, select the visibility required by the instructor, and leave **README**, **.gitignore**, and **license** unchecked because the local project already exists.

From the project root, create the first commit and connect it to the empty GitHub repository. Replace `USERNAME` and `REPOSITORY`:

```
git init -b main
git add .
git status
git commit -m "Add machine-learning course work"

git remote add origin git@github.com:USERNAME/REPOSITORY.git
git remote -v
git push -u origin main
```

Common mistake

Always read `git status` before committing. If a dataset, password, token, `.env` file, or virtual environment appears, stop and update `.gitignore` before running `git commit`. Deleting a secret in a later commit does not remove it from earlier repository history.

1.7.6 Submit later changes

Use short commits as each lab progresses:

```
git status
git add .
git commit -m "Complete Lab 1 feature preparation"
git push
```

Submit the GitHub repository address requested by the instructor and confirm that the latest commit and required notebooks are visible on GitHub.